

Presentation Script — Graph-Based Screening (talk notes)

Presentation Script: Graph-Based Screening for Potentially Cooperative Vessel Pairs

A temporal link prediction approach using AIS (2012–2019)

Total timing: ~20 minutes

(Practice tip: Speak at a natural pace. The deep-dive section on the TGCN is the longest—aim for 8–9 minutes there as requested by the professor. Pause briefly at each figure to let the audience absorb it. Total slides: title + 9 content slides.)

Slide 1: Title (30 seconds)

Good [morning/afternoon], everyone. My name is Oumayma Ben Aoun. I’m a data science student, and my faculty mentor is Dr. Duman. My senior project is titled “Graph-Based Screening for Potentially Cooperative Vessel Pairs.”

Slide 2: Agenda (30 seconds)

In Today’s agenda we’ll cover the problem and motivation, the end-to-end pipeline, how we build the daily graphs, gear-aware stratification, case studies, and finally limitations and conclusion.

Slide 3: Problem & Motivation (2 minutes)

Illegal, unreported, and unregulated—or IUU—fishing costs the global economy between 10 and 23 billion dollars every year. Enforcement agencies desperately need tools that can flag vessels that might be coordinating their behavior, because coordinated fleets are much harder to catch than lone operators.

The Automatic Identification System, or AIS, gives us billions of position records—speed, heading, location—but here’s the catch: cooperation is never labeled. No one has tagged which vessel pairs are actually working together. That means supervised learning is off the table.

Instead, we treat this as an unsupervised temporal link-prediction problem: if two vessels keep appearing near each other across many days, our model should give them a high score. What this work *does* is produce a ranked candidate list with confidence scores for human review. What it *does not* do is prove coordination or hand down verdicts. It’s screening, not sentencing.

To set expectations right away, here’s a quick table on the slide: the model ranks pairs based on learned graph patterns and passes them through independent geographic validation. The output is a shortlist plus reproducible checks—exactly what analysts need.

Slide 4: Pipeline Overview (1 minute)

Here's the full end-to-end pipeline. On the left we start with 2.38 billion vessel-day AIS records spanning 2012 to 2019. Each calendar day becomes one snapshot graph: vessels are nodes, and edges connect vessels that were within 10 km on that same day. We optionally add social edges based on shared national registry prefixes, but we always cap the graph size for tractability.

The TGCN then processes this entire sequence of daily graphs and learns 32-dimensional embeddings for every vessel. We score every possible pair using the inner product of those embeddings and keep the top 200 candidates. Finally, a strict 25 km geographic filter on the raw tracks reduces those 200 to just 8 validated pairs. Gear-aware stratification then explains exactly why the other 192 were filtered out.

Let me walk you through each stage, starting with how we actually construct those daily graphs.

Slide 5: Graph Construction (2 minutes)

We build one graph per calendar day. The training edges—our only signal during learning—are proximity edges: same day, Haversine distance ≤ 10 km between the daily mean positions of two vessels. That's it.

We can also add optional social edges when vessels share the same 3-digit ITU MID prefix—for example, 412 for China—but we cap those at 2,000 per day so the graphs stay manageable on a laptop.

The most important point—and I want to stress this because it makes our validation credible—is that training distances and validation distances are never mixed. The model only ever sees 10 km same-day proximity during training. The 25 km check, plus or minus one day, is applied *after* training on the raw AIS tracks as an independent filter. Training and validation stay completely separate.

Slide 6: Deep Dive: The TGCN Model (8–9 minutes — *emphasize here*)

Now the part the professor asked for: a deep dive into the Temporal Graph Convolutional Network, or TGCN.

At its core, the TGCN combines two classic ideas: a Graph Convolutional Network that learns from who is near whom on a single day, and a Gated Recurrent Unit that learns how those neighborhoods evolve across days. The clever twist is that we replace the GRU's ordinary linear layers with graph convolutions—so the recurrence itself becomes graph-aware. Each vessel's hidden state is updated based on its actual neighbors in that day's graph. We use the TGCN implementation from PyTorch Geometric Temporal by Rozemberczki et al.

Let's break it down. First, recall a standard GCNConv layer, shown on the slide. For each vessel u , we aggregate information from its neighbors:

$$\mathbf{h}_u^{(l+1)} = \sigma \left(\sum_{v \in \mathcal{N}(u)} \frac{1}{\sqrt{d_u d_v}} \mathbf{h}_v^{(l)} \mathbf{W}^{(l)} \right)$$

In plain English: the model looks at every vessel within 10 km today, takes a weighted average of their features (inverse-degree normalized so crowded fishing grounds don't dominate), multiplies by a learnable weight matrix, and applies a non-linearity. After one layer, every embedding already encodes the local neighborhood structure.

Now, inside the TGCN *cell* itself—follow the diagram top to bottom. The inputs are:

- $\mathbf{X_t}$: the 5-dimensional node-feature matrix for today
- $\mathbf{H_t-1}$: the 32-dimensional hidden state carried forward from yesterday

We compute three gates, each using a GCNConv so they’re fully graph-aware:

1. The **update gate** Z_t (sigmoid) decides how much of yesterday’s hidden state to keep.
2. The **reset gate** R_t (sigmoid) decides how much of yesterday to forget before computing the new candidate.
3. The **candidate state** $\tilde{H}_t$ (tanh) is what today’s graph suggests the new hidden state should be.

The final hidden state is the blend:

$$\mathbf{H}_t = \mathbf{Z}_t \odot \mathbf{H}_{t-1} + (1 - \mathbf{Z}_t) \odot \tilde{\mathbf{H}}_t$$

If Z is close to 1, the vessel’s behavior hasn’t changed much and we keep yesterday’s embedding. If Z drops toward 0, today’s graph neighborhood overwrites the memory. That hidden state then becomes H_{t-1} for the next day—pure recurrence, but every gate is graph-aware.

The exact equations for the gates are on the slide (update, reset, candidate, and final blend). Each gate concatenates today’s graph-processed features with yesterday’s hidden state, applies a linear layer and activation, and uses element-wise multiplication so every dimension of the 32-dimensional embedding is gated independently.

What features actually go into X_t ? Five per vessel per day, all z-score standardized:

- Daily degree (neighbors within 10 km today)
- Total interaction count across the whole training period
- Number of unique partners ever seen
- Days since last appearance
- Average gap between appearances

These capture both today’s activity and long-term behavioral profiles.

After the TGCN processes a day, we score every pair with the simple inner product of their embeddings:

$$\text{score}(u, v) = \mathbf{h}_u^\top \mathbf{h}_v$$

High score means the two vessels have learned similar neighborhood patterns across time, so the model predicts they are likely to co-occur again. We rank all pairs and keep the top 200.

Training is fully unsupervised: we process the 1,450 training days in chronological order, treat actual edges as positives, sample the same number of random non-edges as negatives, and use BCEWithLogitsLoss. We detach the hidden state after every day—truncated BPTT—so the 16 GB laptop doesn’t run out of memory. Gradients flow only one day at a time, but the daily recurrence still captures meaningful temporal structure.

On the test set (days 1,451–2,326) we get ROC AUC 0.73 and AP 0.71. That’s moderate but meaningful for an unsupervised model—no leakage from random splits, genuine forecasting ability. The per-bucket standard deviation of ± 0.10 across the 876 test days shows some days are easier than others, which is expected.

Slide 7: Results & Validation (2 minutes)

Let’s look at the quantitative numbers. Our primary setting—proximity plus capped social edges—achieves 0.730 AUC and 0.714 AP. The proximity-only ablation actually does slightly better (0.776/0.795), because the dense Chinese registry cliques add noise. The capped-graph sanity check hits ~ 0.95 , confirming the model works when the task is easy.

Of the top 200 TGCN candidates, we applied the strict 25 km $\pm$ 1-day filter on raw tracks. Only 8 pairs survive. Figure 1 shows our flagship pair—412000690 and 412325200—with 102 days inside 25 km. The blue and orange traces overlap heavily in the Yellow Sea hot spot. If the model were just surfacing random pairs, we would never see this sustained real-world co-presence.

Slide 8: Gear-Aware Stratification (3 minutes)

Why did 192 of the 200 candidates fail the filter? We ran a three-step gear-aware analysis. First, we enriched every MMSI with its dominant gear type using 96 sampled fleet days. Second, we computed a baseline of how often each gear pair normally shares a grid cell—roughly 7 million cell-days. Third, we discounted each TGCN score by that baseline rate ($\alpha = 2$).

The headline result: 75% of the top 200 are trawler-dominated. Trawlers fish the same waters by default, so their graph similarity is mostly fleet-level noise. The pass rates tell the real story: cross-gear pairs survive at 10x the rate of trawler-trawler pairs (25% vs 2.4%).

The 10x Story

75% of top 200 are trawler-dominated — they fish the same waters by default

Pass rates tell the real story:

Fixed gear + Trawlers: 25% (1 of 4 pairs pass)

Set gillnets + Trawlers: 8.3% (1 of 12)

Line / generic + Trawlers: 6.0% (4 of 67)

Trawlers + Trawlers: 2.4% (2 of 82)

10x difference between cross-gear and same-gear pass rates

Figure 2, panel (a), shows all 200 candidates as stacked counts by gear combination—gray segments are pairs that failed the 25 km check, green segments passed—so you can see exactly where the 192 failures pile up. The small strip below sums the whole cohort: 192 failed plus 8 passed. Cross-gear categories still have much higher pass *rates* than trawler-trawler; the bracket calls that out. Panel (b) lists the 8 validated pairs. Six of the eight are cross-gear. The two trawler-trawler survivors have higher mean distances and lower ranks. One set-gillnet + trawler pair even jumped 48 ranks after the gear adjustment—exactly the kind of distinctive signal we want.

Takeaway: the geographic filter is justified, and the 8 validated pairs are enriched for behaviorally unusual encounters.

Slide 9: Case Studies (2 minutes)

Figure 3 shows monthly overlap for all eight pairs, sorted by total days. Pair 1 dominates with large bubbles clustered in 2015–2016—persistent co-presence. Most others are sporadic, just a handful of small bubbles.

Figure 4 plots all rendezvous locations against the Chinese coast. Six vessels appear in multiple pairs and converge on a narrow band 50–80 km offshore from Ningbo (centroid approx. 29.7 deg N, 122.3 deg E). Different pairs, same operating region—another signal that these are not random coincidences.

Slide 10: Limitations & Conclusion (2 minutes)

A few honest limitations: link prediction is not a verdict—high scores can reflect shared fishing grounds or traffic lanes. AIS has coverage gaps. Crowded regions create many possible edges. We report single-seed results with per-bucket dispersion; multi-seed runs are future work. Gear labels are coarse fleet categories, and the adjustment is a heuristic.

In conclusion, we built a fully unsupervised AIS-to-graph-to-TGCN pipeline with a clean time-split evaluation. We achieve 0.73 AUC / 0.71 AP, surface 8 pairs that survive strict geographic screening, and use gear-aware stratification to show that 75% of candidates are normal fleet behavior while cross-gear pairs survive at 10x the rate. The output is a ranked candidate list plus reproducible checks—not a determination of wrongdoing.

Future work includes multi-seed runs on bigger hardware, heterogeneous-edge architectures that weight proximity versus registry edges differently, finer resolution, and richer registry metadata.

Thank you. I'm happy to take any questions—feel free to flip back to Figure 2 or the TGCN diagram if you'd like to discuss any part in more detail.